Stocarea datelor, modalitati de acces

MEMORAREA DATELOR, FISIERE, ORGANIZARE

Mediul de stocare poate fi impartit in doua categorii:

- *stocarea primara* priveste acel mediu asupra caruia se poate opera direct de catre unitatea centrala (memoria interna).
- *stocarea secundara* se refera la toate mediile externe de memorare, cum sunt discul magnetic, banda magnetica, disc optic, caracterizate de capacitate mare, cost redus, dar cu un timp de acces mult mai mare decit memoria interna.

Justificarea stocarii externe a BD:

- volum prea mare pentru a fi stocate in memoria principala;
- datele trebuie sa ramana dupa intreruperea sursei de alimentare, adica au caracter permanent;
- costul pe unitatea de informatie este mult mai mic;
- poate fi asigurata transportabilitatea intre mai multe masini

Disc magnetic (organizare)

- Informatia este stocata pe suprafata discului pe cercuri concentrice cu diametre distincte numite *piste* (*track*). La discurile compacte pistele de aceeasi raza formeaza un *cilindru*.
- Fiecare pista este impartita in mai multe *blocuri*, ce apartin unui *sector*. Marimea unui bloc este cuprinsa uzual intre 512 si 4096 octeti functie de sistemul de operare aferent. Separarea blocurilor se face in spatii de lungime fixa prin *gap interbloc*, ce contine informatii speciale de control scrise in momentul initializarii (formatarii) discului.
- Orice referire la informatiile stocate pe disc este o referire la un bloc. Adresa blocului este obtinuta prin combinarea indicativului suprafetei, indicativului pistei (intr-o suprafata) si indicativului blocului (in cadrul unei piste).
- Accesul la informatiile de pe disc se face prin citirea unui bloc si aducerea acestuia in memoria principala intr-o zona contigua numita si *buffer de intrare-iesire (I/O)*.
- Cand mai multe blocuri se transfera in memoria interna, existenta mai multor buffere face ca viteza de transfer sa creasca.

- Daca blocurile sunt contigue (alocare contigua) se consuma un timp de 15 - 60 ms pentru accesul la primul bloc, celelalte blocuri citindu-se in 1 -2 ms fiecare.
- Sistemul de calcul utilizat in baze de date are performante cu atit mai bune cu cat timpul de acces la informatia stocata pe disc este mai mic.
- Cand mai multe blocuri se transfera in memoria interna, existenta mai multor buffere face ca viteza de transfer sa creasca.
- In timpul citirii sau scrierii unui bloc unitatea centrala poate procesa alt bloc deja transferat in memoria interna.
- Prelucrarea paralela este posibila datorita existentei procesorului I/O pentru disc, ce poate lucra independent de unitatea centrala.

Obs. Din aceasta scurta descriere rezulta ca performantele oricarei aplicatii de baze de date sunt afectate de accesul la memoria externa.

Concluzie: Este foarte important ca numarul operatiilor de I/O pentru accesul la date sa fie cat mai redus.

Alocarea blocurilor

- Alocare contigua (blocuri de pe acelasi cilindru sau cilindrii vecini)
- Alocarea cu legaturi (pentru stocarea fisierelor secventiale in sisteme de operare) – FAT
- Alocare indexata (cu blocuri index pe diverse niveluri – structura arborescenta) metoda caracterizata de existenta unor pointeri de inlantuire a blocurilor
- Alocare bazata pe HASH Pointer = functie_hash(camp_hash) si indica grupul in care se gaseste inregistrarea. Utilizata atat la stocare cat si la citire

- Inregistrari cu lungime fixa



- Inregistrari cu lungime variabila



- In general, un bloc contine mai multe inregistrari, dar pot exista inregistrari ce incep intr-un bloc si se termina in alt bloc, functie de dimensiunea inregistrarii si a blocurilor.

- Daca se noteaza numarul de octeti ai unui bloc cu B, si cu R numarul de octeti ai unei inregistrari (valabil doar pentru inregistrarile de lungime fixa), numarul inregistrarilor ce pot fi stocate intr-un bloc va fi dat de realtia:

  bfr = [B/R] in care,

- bfr - reprezinta *numarul de inregistrari pe bloc (blocking factor rate)*
- [x] - reprezinta functia parte intreaga.
- In general R nu divide pe B, deci, daca nu sint inregistrari ce trec dintr-un bloc in altul, in fiecare bloc folosit ramine un spatiu neocupat de B-(bfr*R) octeti.

Obs:

1. O tehnica uzuala este definirea unor blocuri logice formate din mai multe blocuri fizice contigue.
2. Blocurile nu sunt complet ocupate

Metode de cautare

1. Cautare liniara (secventiala) specifica cautarii dupa chei de acces neordonate
2. Cautare dihotomica. (cautare prin segmentare – specifica cautarii dupa chei de acces ordonate)

- Cautare binara. Se considera un fisier secvential cu N inregistrari, ale carui chei sint in ordine crescatoare k1 < k2 <....< kN, cautarea avind ca scop gasirea inregistrarii cu cheia k.
- Cautare binara uniforma
- Cautare Shar (nod radacina [$\log_2 n$]
- Cautare Fibonacci (nodurile sunt termeni ai sirului Fibonacci)

Operatii la tabele de date

- ***Find (Locate***) operatie de cautare a primei inregistrari ce satisface un criteriu de acces specificat.
- ***Read (Get)*** operatia de copiere din buffer intr-o variabila program sau zona de lucru pentru utilizator a unei inregistrari.
- ***Find next*** operatie de cautare a urmatoarei inregistrari ce satisface un criteriu de selectie specificat.
- ***Delete*** stergere a inregistrarii curente sau a grupului de inregistrari ce satisfac criteriul de selectie, si eventual rearanjarea tabelei pe disc pentru a reflecta aceasta modificare..
- ***Modify (Update)*** operatie de modificare a valorilor campurilor din inregistrarea curenta sau din inregistrarile ce satisfac criteriul de selectie.

- ***Insert*** este operatia de adaugare a noi inregistrari in tabela de date, localizand blocul unde inregistrarile vor fi inserate, transfera blocul in bufferul intern (daca nu este aici), inscrie inregistrarea in buffer, scrie bufferul pe disc pentru a reflecta aceasta modificare. De foarte multe ori operatia de insert este inlocuita cu append, operatie mult mai simpla ce adauga o inregistrare la sfarsitul tabelei.
- ***Sort*** operatie de ordonare fizica a tuturor inregistrarilor din tabela in ordinea specificata prin criteriul de selectie.
- ***Index*** este operatia de creare a unei ordini logice alta decat cea fizica. Operatia creaza structuri de date aditionale pe langa tabela de date, structuri ce definesc ordinea logica.

Trebuie facuta distinctia neta intre organizarea tabelei si metoda de acces. Prin organizare tabelei (table organization) se intelege organizara datelor in inregistrari, blocuri si structuri de acces. Prin metoda de acces (acces metod) se intelege grupul de programe ce permit aplicarea operatiilor de mai sus asupra tabelei. In general metodele de acces sunt dependente de organizarea tabelei

Fisiere de date

Se cunosc doua moduri de organizare a informatiilor in fisier:

- organizare definita (fisier logic)
- organizare nedefinita (fisier fizic).. Accesul la un fisier cu organizare nedefinita are ca obiect obtinerea unor inregistrari fizice.

Principalele cerinte ale sistemelor de baze de date pot fi sintetizate astfel:

- stocarea unor cantitati mari de date;
- accesul usor la date;
- economia de spatiu la stocarea informatiilor;
- fiabilitatea si protectia datelor;
- capacitatea de reprezentare a structurilor lumii reale.

Concluzie: Alegerea metodei de organizare va determina performantele relative ale sistemului.

Pentru a judeca calitativ performantele se analizeaza de regula urmatoarelor aspecte:

- spatiul necesar pentru a pastra o inregistrare;
- timpul necesar pentru accesul la o inregistrare arbitrara;
- timpul necesar pentru accesul la urmatoarea inregistrare;
- timpul de stocare dupa inserarea unei noi inregistrari;
- timpul de reorganizare dupa adaugarea unei noi inregistrari;
- timpul de citire a intregului continut.

Tehnica Hash

- O metoda de organizare primara ce asigura un acces rapid la inregistrari dupa criterii specificate este cea utilizind tehnica *hash*.
- Se foloseste si termenul de acces *direct*, indicand accesul relativ prin pozitia inregistrarii.
- Criteriul de cautare este dat de egalitatea conditiei la un singur camp numit si *camp hash*, care nu este obligatoriu un camp cheie, numit si *cheie hash*.
- Inregistrarile sunt organizate in grupuri ce ocupa unul sau mai multe blocuri. Functia hash aplicata cimpului hash produce un intreg ce indica adresa blocului in care inregistrarea dorita este stocata, daca aceasta exista.
- In functie de locul in care se aplica tehnica hash, se intalnesc o serie de variante numite: hash intern (internal hash) pentru fisiere temporare in memoria interna, hash extern (external hash) pentru fisiere stocate pe disc.

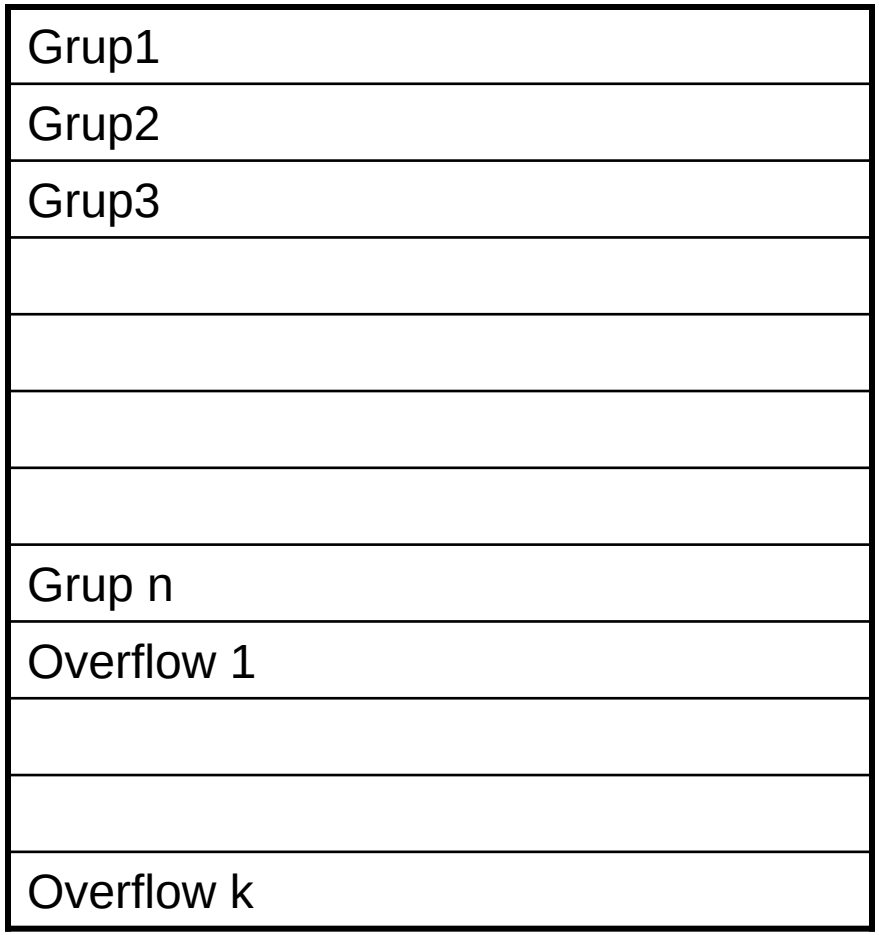
Hash intern

Se aplica la stocarea si accesul la date din memoria interna pentru care se utilizeaza un spatiu aprioric alocat pentru dispunerea inregistrarilor. Daca consideram marimea spatiului alocat pentru stocarea datelor in memoria interna este cuprins intre 0 si H-1, atunci spunem ca exista H sloturi. Vom alege o functie hash care transforma valoarea cimpului hash intr-un intreg cuprins intre 0 si H-1.

Nu exista o modalitate stricta privind alegerea functiei hash, insa dintre cele mai utilizate metode amintim:

- retinerea unui numar fixat de biti din valoarea campului hash;
- obtinerea restului impartirii valorii intregi a cimpului hash considerat ca o secventa de biti la H;
- aplicarea unei functii injective pe anumite portiuni ale cimpului hash considerat ca sir de biti.

Exemplu


Grup1
Grup2
Grup3
Grup n
Overflow 1
Overflow k
Grupuri initiale (date de valorile functiei hash)
Grupuri overflow

Hash extern

- O tehnica hash similara se aplica si asupra datelor stocate pe disc, pentru a imbunatatii accesul la blocurile de date.
- Metoda hash aplicata la datele stocate pe suport extern va fi adaptata la caracteristicile mediului de stocare.
- Blocurile sunt grupate dupa o anumita strategie, un bloc sau un grup de blocuri formeaza un grup (cluster).
- Pentru accesul la disc in loc sa se specifice adresa inregistrarii la fiecare operatie se va specifica adresa clusterului sau adresa relativa in cluster.
- Functia hash indica un numar relativ al cluster-ului, ce poate fi utilizat la calculul adresei absolute.
- Adresa blocurilor disc poate fi pastrata intr un fisier header.
- La coliziuni o metoda de rezolvare este inspirata de la tehnica hash interna, prin inlantuirea de blocuri overflow.
- la fiecare cluster se adauga un pointer ce inlantuie inregistrarile cu aceeasi functie hash plasate intr-un bloc overflow.

Tipuri de indecsi

- Index primar. Un index primar este realizat pe baza unui atribut cheie la o tabela ordonata dupa atributul (campul) cheie.
- Indexul secundar poate fi construit dupa un camp cheie al tabelei de date, insa tabela nu este ordonata dupa acest camp.
- Index de grup. Daca campul de indecsare nu este un camp cheie, putand avea pentru mai multe inregistrari aceeasi valoare structura aditionala de acces poarta denumirea de index de grup (clustering index).

La o tabela de date se pot construii oricate structuri de index functie de atributele utilizate pentru constructia sa.

- Index multinivel atunci cand indexul se trateaza similar cu o tabela la care se construieste in continuare index. Exista mai multe metode de implementare pentru index multinivel prin arbori B sau arbori B+.

Obs. Trebuie inteleasa diferenta majora intre indexare si sortare. Indexarea este o metoda de acces logic pe cand sortarea se refera la reorganizarea tabelei si produce la o alta ordine fizica.

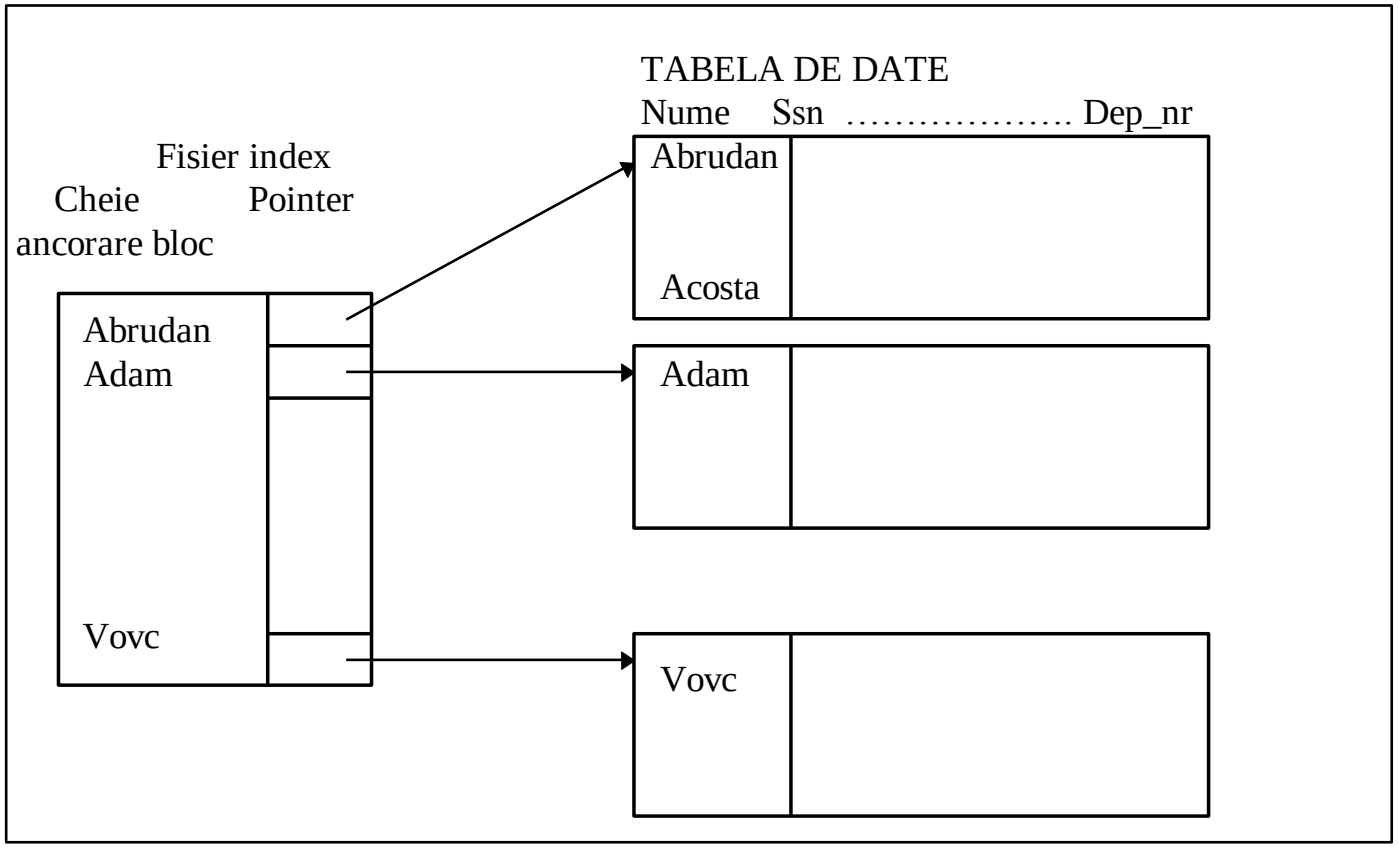
Index primar

- Un index primar este un exemplu de index *nondens* datorita faptului ca are cate o intrare pentru fiecare bloc, fata de indexul *dens* ce asociaza cate o intrare pentru fiecare inregistrare a tabelei de date.

Indexul primar necesita mai putine blocuri pentru stocare datorita faptului ca;

- asociaza cate o singura intrare (inregistrare) unui bloc de date;
- o intrare index ocupa mult mai putini octeti decat o inregistrare a tabelei de date. Ca efect cautarea intr-un index este mult mai rapida decat intr-o tabela de date, putind fi utilizate si metode de cautare binara deoarece indexul este totdeauna ordonat.
- O inregistrare pentru care valoarea cheii primare este k se gaseste in blocul i daca este indeplinita conditia k(i)<=k<k(i+1). Adresa blocului i este P(i) si inregistrarile in acest bloc sunt ordonate fizic dupa cimpul cheie.

Exemplu de index primar



Calcul performante (exemplu)

- Consideram o tabela de date ordonata dupa un camp cheie, ce contine r=30000 inregistrari.
- Tabela este stocata pe disc in blocuri avand dimensiunea B=1024 octeti.
- Toate inregistrarile sunt de lungime fixa si o inregistrare ocupa R=100 octeti.
- Numarul de inregistrari pe bloc se calculeaza ca fiind bfr=[B/R]=[1024/100], obtinand bfr=10. Nu este o pierdere faptul ca in fiecare bloc raman neutilizati 24 octeti, mult mai importante sunt performantele.
- Numarul de blocuri pentru tabela de date b=[r/bfr]=[30000/10]=3000, eventual plus 1. .

- Cautarea binara in tabela de date necesita aproximativ $[\log_2 b]+1$ operatii de citire blocuri, deci $[\log_2 3000] + 1=12$ blocuri, numar de blocuri destul de mic rezultat din faptul ca tabela este ordonata.

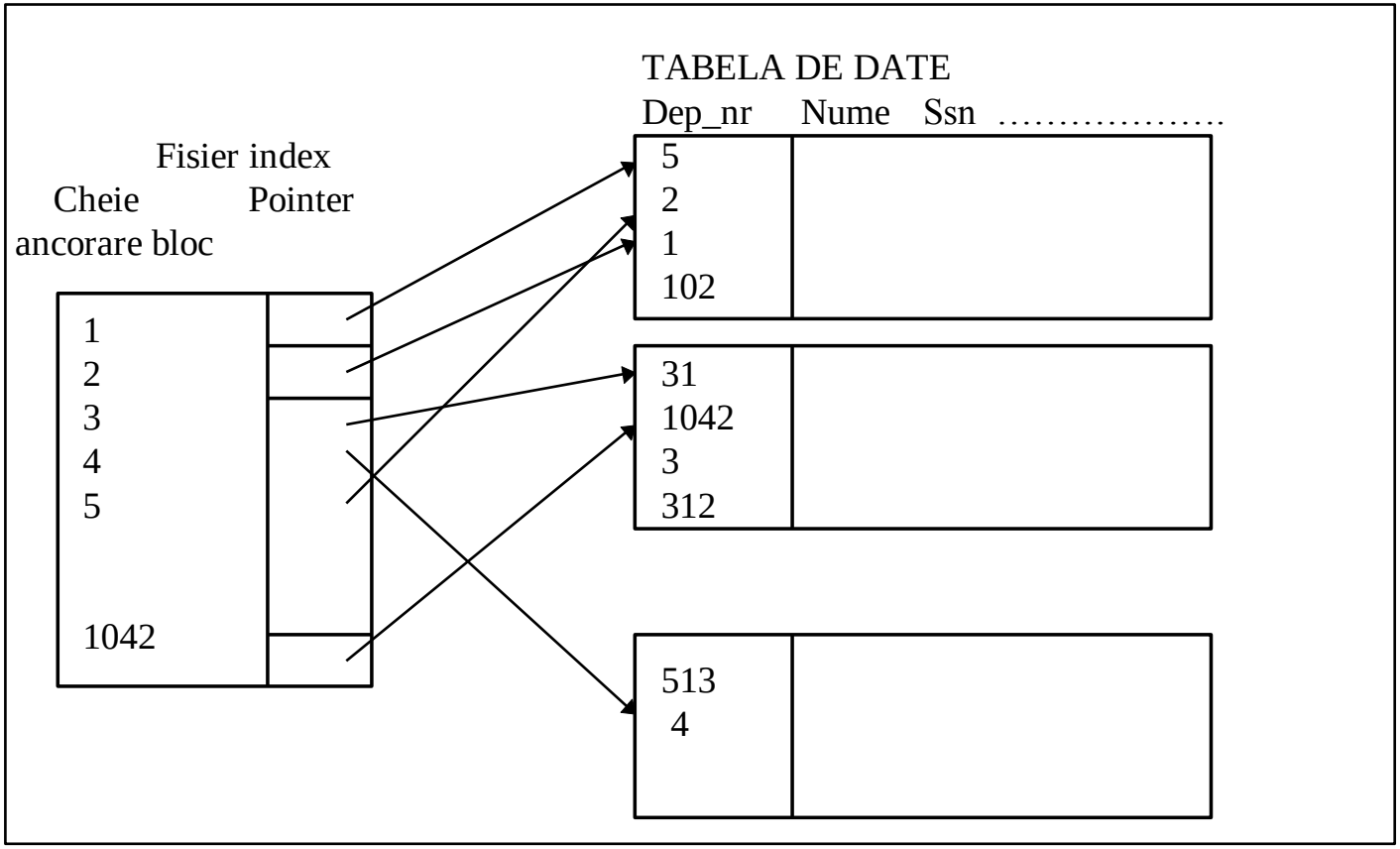
Acces cu index

- lungimea campului cheie de ordonare a tabelei este de 9 octeti si ca un pointer catre un bloc poate fi stocat pe 6 octeti. O intrare index este $R_i=V+P=9+6=15$ octeti.
- Numarul intrarilor index pe bloc devine $bfr_i=[B/R_i]=[1024/15]=68$
- Numarul de intrari index este dat de numarul blocurilor tabelei de date, in cazul de fata $r_i=3000$. Numarul de blocuri index $b_i=[r_i/bfr_i]+1=[3000/68]+1=46$.
- Cautarea binara in index necesita aproximativ $[\log_2 b_i]+1+1$ operatii de citire bloc, adica $[\log_2 46]=5+1+1$. In total pentru a se gasi inregistrarea va fi necesara citirea a 7 blocuri, 6 apartinand indexului si unul apartinand tabelei de date.

Index secundar

- Indexarea secundara se aplica la tabele neordonate dupa campul de indexare.
- *Indexul secundar* este o structura ordonata cu doua coloane, in care prima coloana este identica cu un camp al tabelei de date (camp de indexare), al doilea camp este un pointer catre blocul tabelei de date.
- Campul pentru care indexul este construit se numeste si *camp de indexare*. In concluzie, orice camp al unei tabele poate fi camp de indexare pentru indexul secundar.
- Un index secundar construit dupa un camp cheie din tabela de date. creaza o intrare index pentru fiecare inregistrare din tabela de date. Se obtine astfel un index dens, avand cate o intrare pentru fiecare inregistrare din tabela primara. Desigur ca, valorile din coloana pointer sunt neordonate, indicind de multe ori acelasi bloc intrucat intr-un bloc se pot gasi mai multe inregistrari.

Index secundar (tabela neordonata)



Index de grup (cluster)

Astfel de indexi sint folositi cind inregistrarile tabelei de date sint ordonate fizic dupa un cimp noncheie, sau neordonate dupa un camp noncheie.

Tabela ordonata

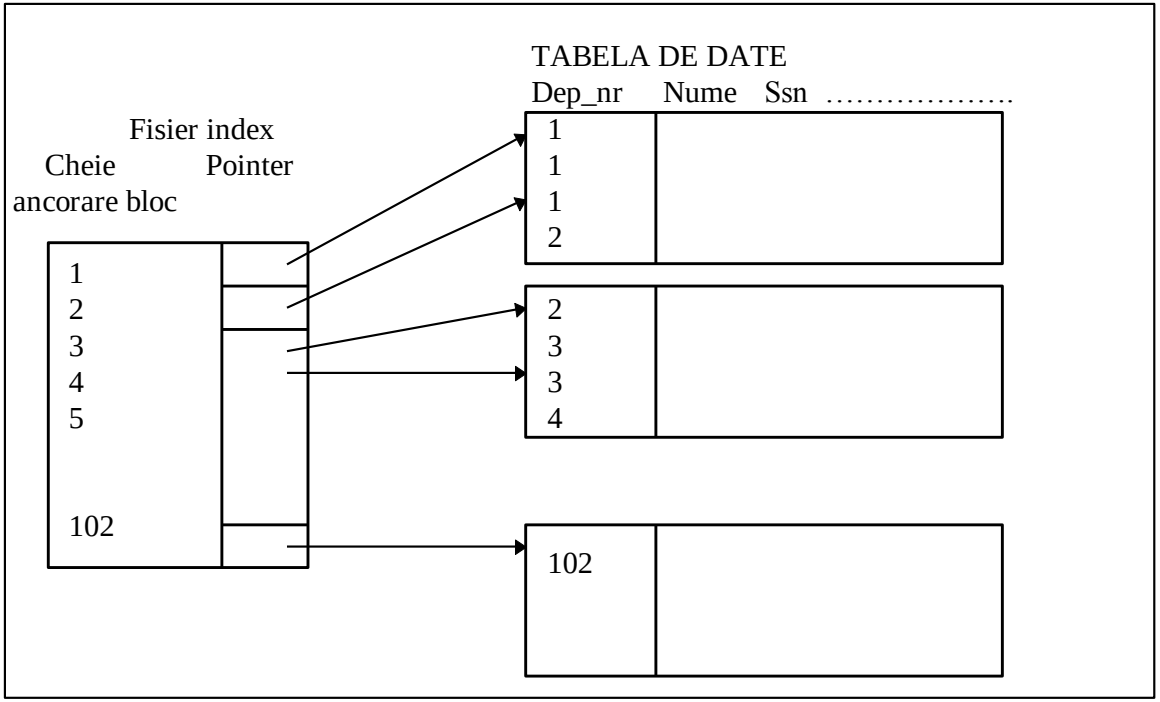
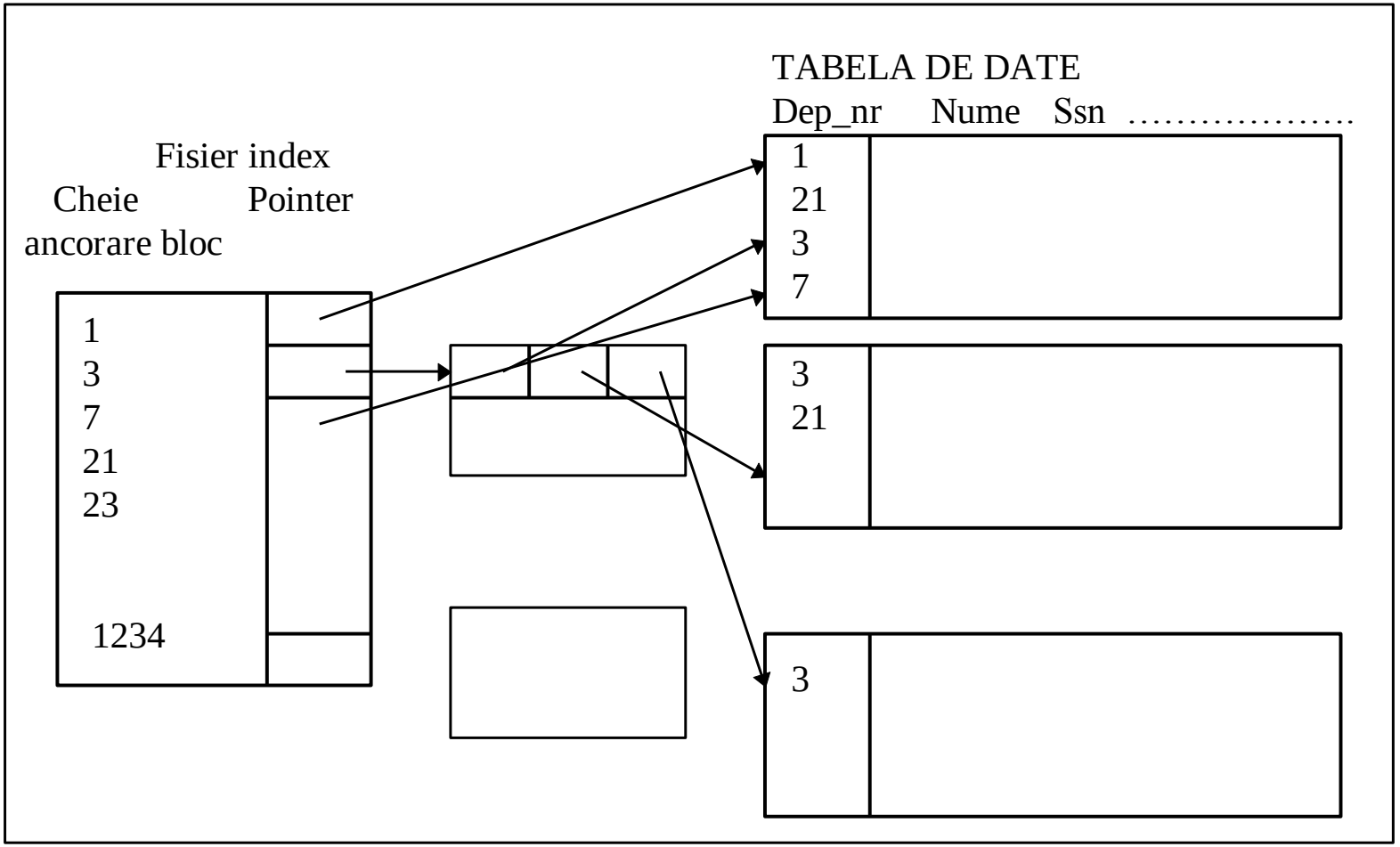


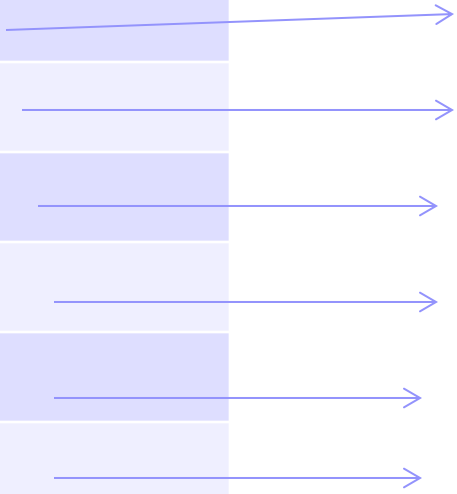
Tabela nordonata



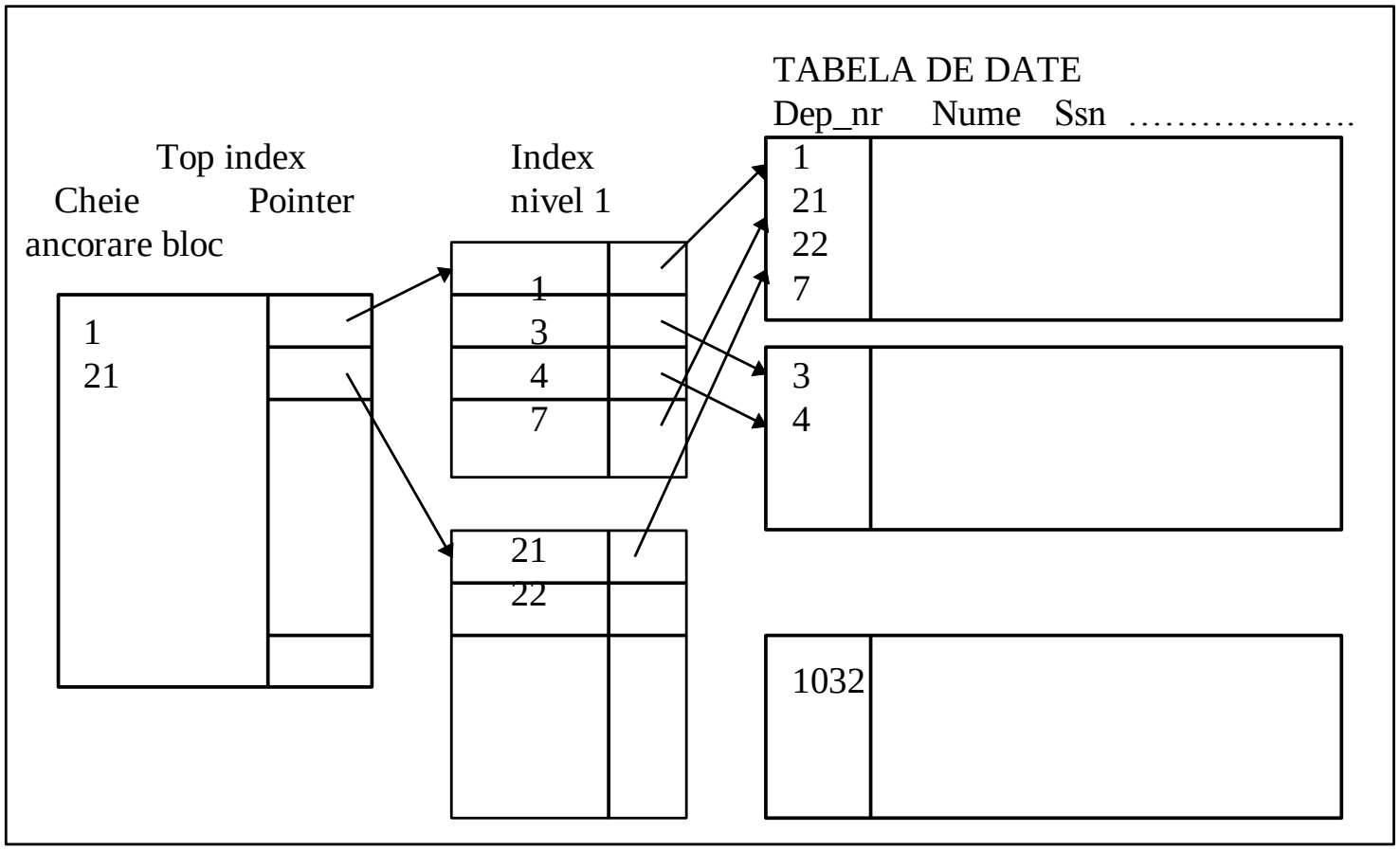
Obs:

- Este mult mai rezonabil sa se pastreze intrari multiple in index decat inlantuirea cu blocuri de pointeri.

Camp indexare	Pointer
1	
2	
2	
2	
3	
3	



Index multinivel (arbori B, arbori B+)



Arbori B

Un arbore B de ordin p, construit dupa un cimp cheie al fisierului index satisface urmatoarele restrictii:

1. Fiecare nod intern in arborele B are structura

<P1,<K1,Pr1>,P2,<K2,Pr2>,...,<Kq-1,Prq-1>,Pq>

cu q<=p, in care, Pi este un pointer la un nou nod arbore numit si *pointer arbore*, pointerii Pri fiind *pointeri data*, pointeri catre blocuri in tabela de date ce contin inregistrarea cu cheia k.

2. In fiecare nod K1<K2<....<Kq-1

3. Valoarea inregistrarii cu cheia X in subarborele punctat de Pi este data de conditiile Ki-1<X<Ki pentru 1<i<q, X<Ki pentru i=1 si Ki-1<X pentru i=q

4. Fiecare nod are cel mult p pointeri arbore

5. Fiecare nod exceptind radacina si nodurile frunza au cel putin [p/2] pointeri arbore. Nodul radacina are cel putin doi pointeri.

6. Un nod cu q pointeri arbore, q<=p, contine q-1 valori ale cimpului de indexare, deci q-1 pointeri data

7. Toate nodurile frunza sunt la acelasi nivel, au aceeasi structura exceptind faptul ca toti pointeri arbore Pi sunt nuli.

Arbori B+

1. Fiecare nod intern are structura

<P1,K1, P2,K2,...,Pq-1,Kq-1, Pq>

unde q<=p si fiecare Pi reprezinta un pointer arbore.

2. In fiecare nod este indeplinita conditia

K1<K2<....<Kq-1, q<=p

3. Pentru toate valorile cimpului de cautare X in subarborele punctat prin Pi se indeplineste relatia

Ki-1<X<=Ki pentru 1<i<q, X<=Ki pentru i=1 si Ki-1<X pentru i=q

4. Fiecare nod intern are cel mult p pointeri arbore.

5. Fiecare nod intern exceptind radacina are cel putin [p/2] pointeri arbore. Nodul radacina are cel putin doi pointeri arbore daca este un nod intern si q<=p pointeri data daca este nod frunza.

6. Un nod intern cu q pointeri, q<=p, are q-1 valori ale cimpului de cautare.

Pentru nodurile frunza sint indeplinite urmatoarele restrictii

1. Fiecare nod frunza este de forma

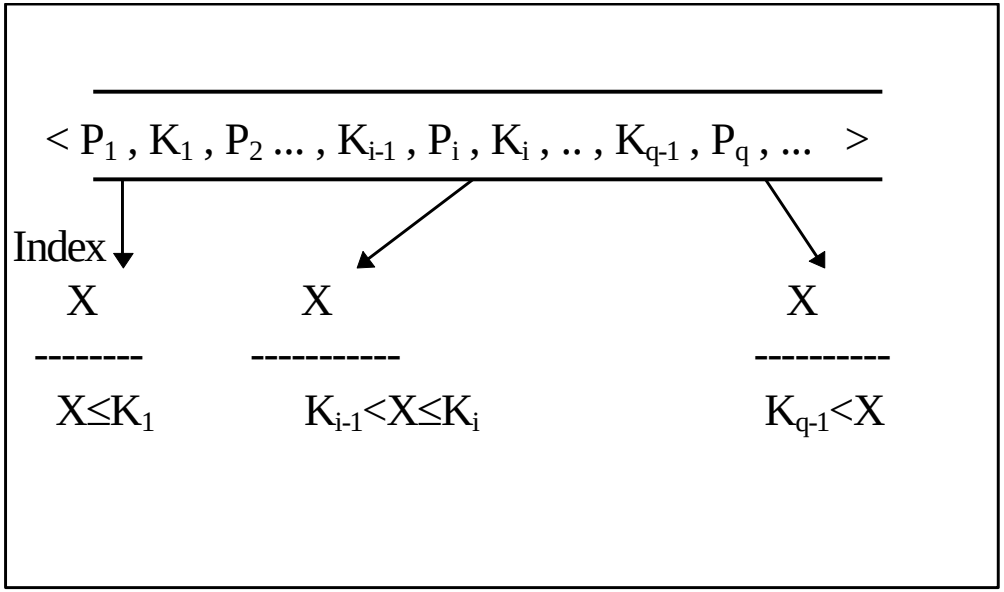
<<K1,Pr1>,<K2,Pr2>,...,<Kq-1,Pq-1>,Pnext>

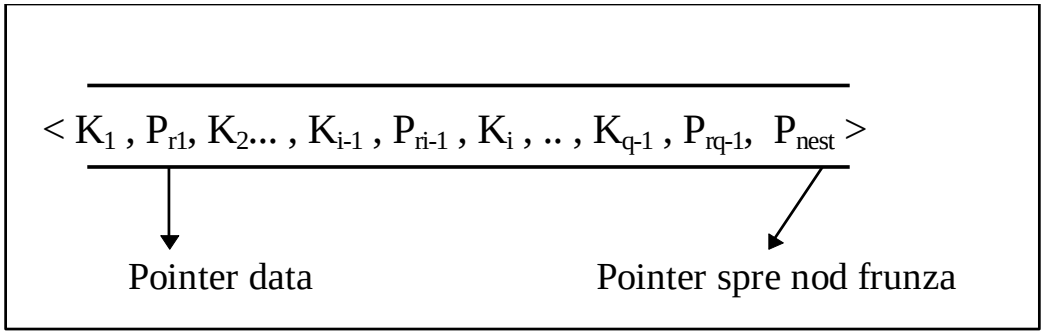
unde q<=p, fiecare Pri este un pointer data si Pnext indica urmatorul nod frunza al arborelui B+.

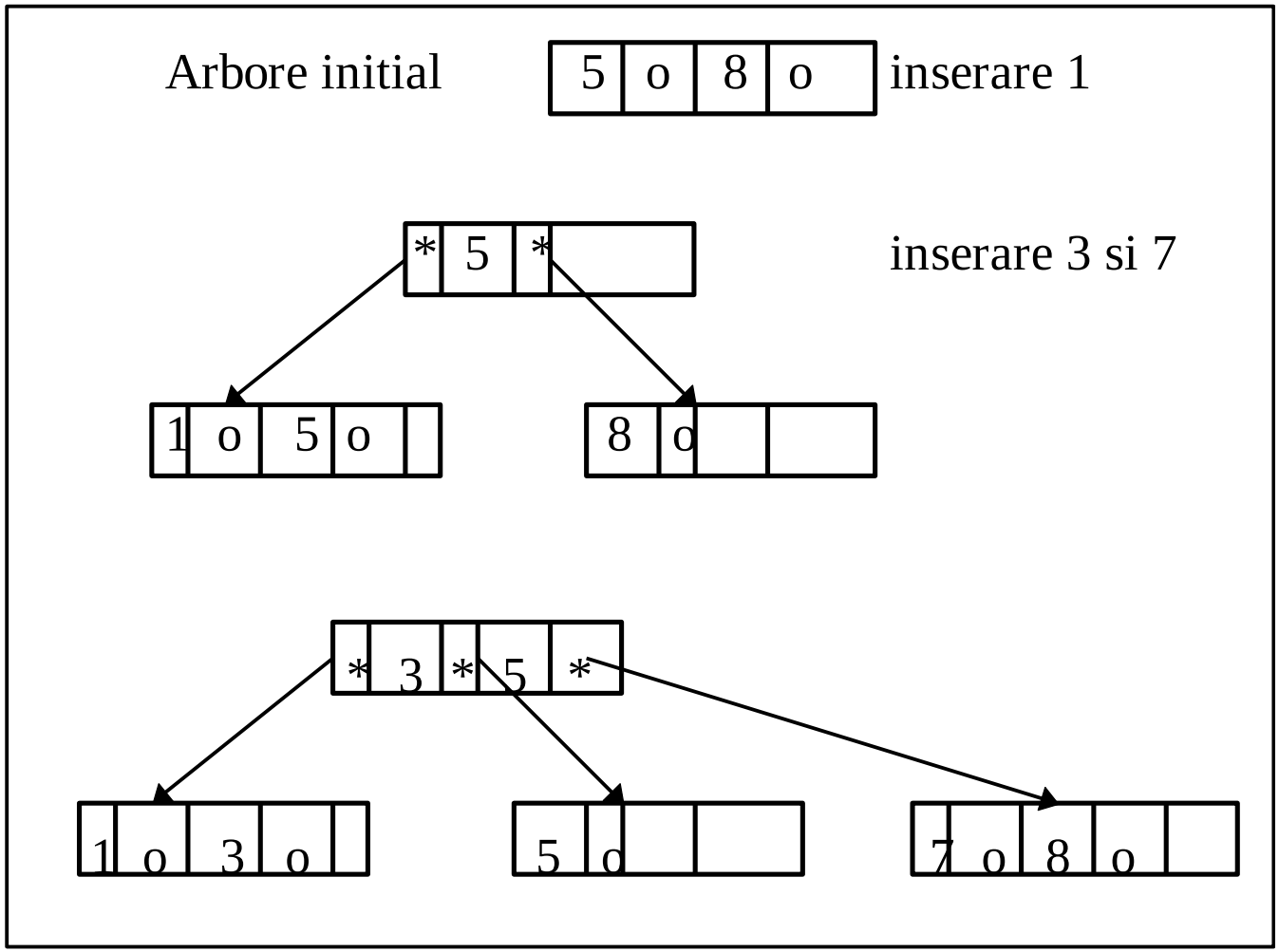
2. In fiecare nod frunza este indeplinita conditia

K1<K2<,...,<Kq-1, q<=p

Arbore B+


< P1 , K1 , P2 ... , Ki-1 , Pi , Ki , .. , Kq-1 , Pq , ... >
Index
X
X
X
X≤K1
Ki-1<X≤Ki
Kq-1<X


< K1 , Pr1, K2... , Ki-1 , Pri-1 , Ki , .. , Kq-1 , Prq-1, Pnest >
Pointer data
Pointer spre nod frunza


Arbore initial
5 o 8 o
inserare 1
* 5 *
inserare 3 si 7
1 o 5 o
8 o
* 3 * 5 *
1 o 3 o
5 o
7 o 8 o